

Se detalla a continuación el planteo y modo de uso de ambos dispositivos I2C utilizados en este proyecto; La memoria EEPROM y el display LCD2004.

Para el display LCD2004, se utilizó una placa conversora PCF8574 que convierte señales I2C en señales aptas para ser recibidas por los 8 pines del display. Cuando se envía 1 Byte (8 bits) por el I2C, se envía cada bit a las terminales P0 a P7 del LCD.

De estas, las primeras 4 se usan para opciones y manejo general:

- P0: Register Select (En 0 indica comando, en 1 indica datos)
- P1: Read/Write (Generalmente en 0 para escribir)
- P2: Enable (Para el clock, detallado mas adelante)
- P3: Backlight (En general siempre en 0 para mantenerla prendida)

Las siguientes 4 terminales (P4-P7) se usan para enviar datos o comandos.

Dado que se deben enviar bytes de 8 bits y solo hay 4 terminales donde mandar bits, se emplea un mecanismo para dividir el bit a enviar en partes. Esto se hace con la función

```
static bool lcdMandarInterno(char data, uint8_t rs)
```

Esta función recibe un byte a enviar al LCD, y un rs que indica si el dato es un comando o un dato a imprimir. Divide el byte en 2 partes, y convierte cada parte en los primeros 4 bits de un byte. Luego envía al I2C 4 bytes: 2 para cada fragmento. Se envían 2 bytes por fragmento porque la lectura del LCD es de flanco descendente: Primero se envía el fragmento con el bit enable en 1, y luego se envía con el bit enable en 0. Finalmente, si la transmisión devuelve HAL_OK, devuelve true. Caso contrario, devuelve false.

El resto de funciones del LCD se basan en el uso de lcdMandarInterno:

```
static bool lcdMandarComando(char cmd);
```

Envía un dato que se toma como argumento, el cual será interpretado como comando (Setea rs en 0). Devuelve false si hay un error en la transmisión, y true si no lo hay

```
static bool lcdMandarDato(char data);
```

Envía un dato, el cual será interpretado como dato a imprimir (Setea rs en 1). Devuelve false si hay un error en la transmisión, y true si no lo hay

```
void lcdBorrar(void);
```

Limpia la pantalla del LCD, haciendo uso del comando 0x01.

```
void lcdSetearCursor(uint8_t col, uint8_t fil);
```

Pone el cursor en la fila y columna indicadas, para luego poder imprimir en esa posición. Se presupone que fil y col son valores que no sobrepasan la cantidad máxima de caracteres del LCD, pero no mueve el cursor si no es así.

```
void lcdPrint(char *cadena);
```

Imprime una cadena, empezando desde la posición del cursor actual. Se presupone que la cadena no sobrepasa el límite máximo de caracteres del LCD.

```
void lcdInicializar(I2C_HandleTypeDef *hi2c);
```

Inicializa el LCD. Primero lo setea a modo 8 bits 2 veces con el comando 0x30, y luego lo setea a modo 4 bits con el comando 0x32. Esto es sugerido por el fabricante porque el chip no sabe en qué modo inicia al encenderse.

Luego, se pone en modo de 4 líneas con el comando 0x28, y enciende la pantalla y esconde el cursor con el comando 0x0C.

Finalmente, borra lo que esté previamente impreso en el LCD, y setea el cursor de manera que avance a la derecha luego de imprimir cada caracter.

```
void lcdBufferearLinea(uint8_t linea, char* cadena);
```

Esta es la función que se llama desde la etapa de procesado. Pone una línea en el buffer, la cual luego puede imprimirse en la etapa de actuadores. Si se bufferea sobre una línea no vacía, esta simplemente se rellena con espacios, sobreescribiendo la línea anterior sin problemas. Una vez buffereado, levanta la flag refreshLCD para que el actuador pueda luego actualizarse.

```
void lcdVaciarBuffer();
```

Como la función anterior, modifica el buffer del lcd. En este caso, llena el buffer con espacios. Esto se hace en lugar del comando borrar porque el comando es bloqueante, y tarda mucho más que esta alternativa. Además, requiere normalmente una espera de 2 ms luego de su uso. Una vez buffereado, levanta la flag refreshLCD para que el actuador pueda luego actualizarse.

```
void lcdActuar();
```

Se llama en el actuador, toma los datos almacenados en el buffer, y los imprime en el lcd. Sólo actúa si el bus I2C no está ocupado, y si la flag de refresh está activada (True).

De lo contrario, no muestra nada y no cambia la flag, permitiendo así que se revise en el siguiente ciclo si se está en condiciones de imprimir. Si imprime algo exitosamente, baja la flag refreshLCD y la setea en false.

Se procede a la documentación de la EEPROM

Para la memoria EEPROM, se utilizó el integrado AT24C256 que permite almacenar datos de manera no volátil. La comunicación se hace mediante I2C, usando direcciones internas de memoria de 16 bits (I2C_MEMADD_SIZE_16BIT).

Dado que se deben escribir bloques de datos de tamaño variable y la memoria no puede escribir atravesando un límite de página de 64 bytes, los datos a enviar se dividen en fragmentos, manteniéndolos por debajo del espacio en la página actual.

```
HAL_StatusTypeDef memEscribir(uint16_t addr, uint8_t *datos, uint16_t largo);
```

Recibe la dirección de memoria addr donde se empieza a escribir, el puntero a los datos y la cantidad de bytes a guardar.

```
HAL_StatusTypeDef memLeer(uint16_t addr, uint8_t *datos, uint16_t largo);
```

Lee una cantidad de bytes indicada por largo desde la dirección addr y los guarda en el puntero de salida `uint8_t *datos`. La lectura completa se hace en un solo llamado a HAL_I2C_Mem_Read, ya que la operación de lectura en la EEPROM no tiene la restricción de límite de página.

```
HAL_StatusTypeDef memLeerMatriz(uint16_t addr, Matriz_t* matriz);
```

Carga en la estructura matriz los datos previamente guardados en la EEPROM desde la dirección addr. Hace uso de memLeer, funcionando como un wrapper para mayor comodidad de uso fuera del TDA. Si la memoria está escribiendo actualmente, devuelve HAL_BUSY. Caso contrario, devuelve un HAL_StatusTypeDef ligado a un llamado de HAL_I2C_Mem_Read.

```
bool memBufferearEscrituraMatriz(uint16_t addr, Matriz_t* matriz);
```

Manda la matriz a guardar al buffer de la memoria, y setea a 0 los valores necesarios para su guardado. Levanta la bandera memEscribiendo para la fase actuar.

Devuelve true si tiene éxito en bufferear la escritura, y false si ya se estaba escribiendo o la matriz a guardar no es válida.

```
bool memEstaOcupada(void);
```

Devuelve el valor de la flag memEscribiendo.

```
void memActuar(void)
```

Si la flag memEscribiendo es false, o si no pasaron al menos 5ms de la última vez que se guardó una página, no hace nada. Caso contrario, calcula cuántos bytes se pueden escribir dentro de la página actual antes de chocar con el límite de 64 bytes y envía ese fragmento mediante HAL_I2C_Mem_Write. Luego actualiza las direcciones para repetir el proceso en el próximo ciclo.